



AthenaStudies

Logic & Sets Day 1

Crash course | Mara Spadon | 2024

Welcome!



Mara Spadon

About me

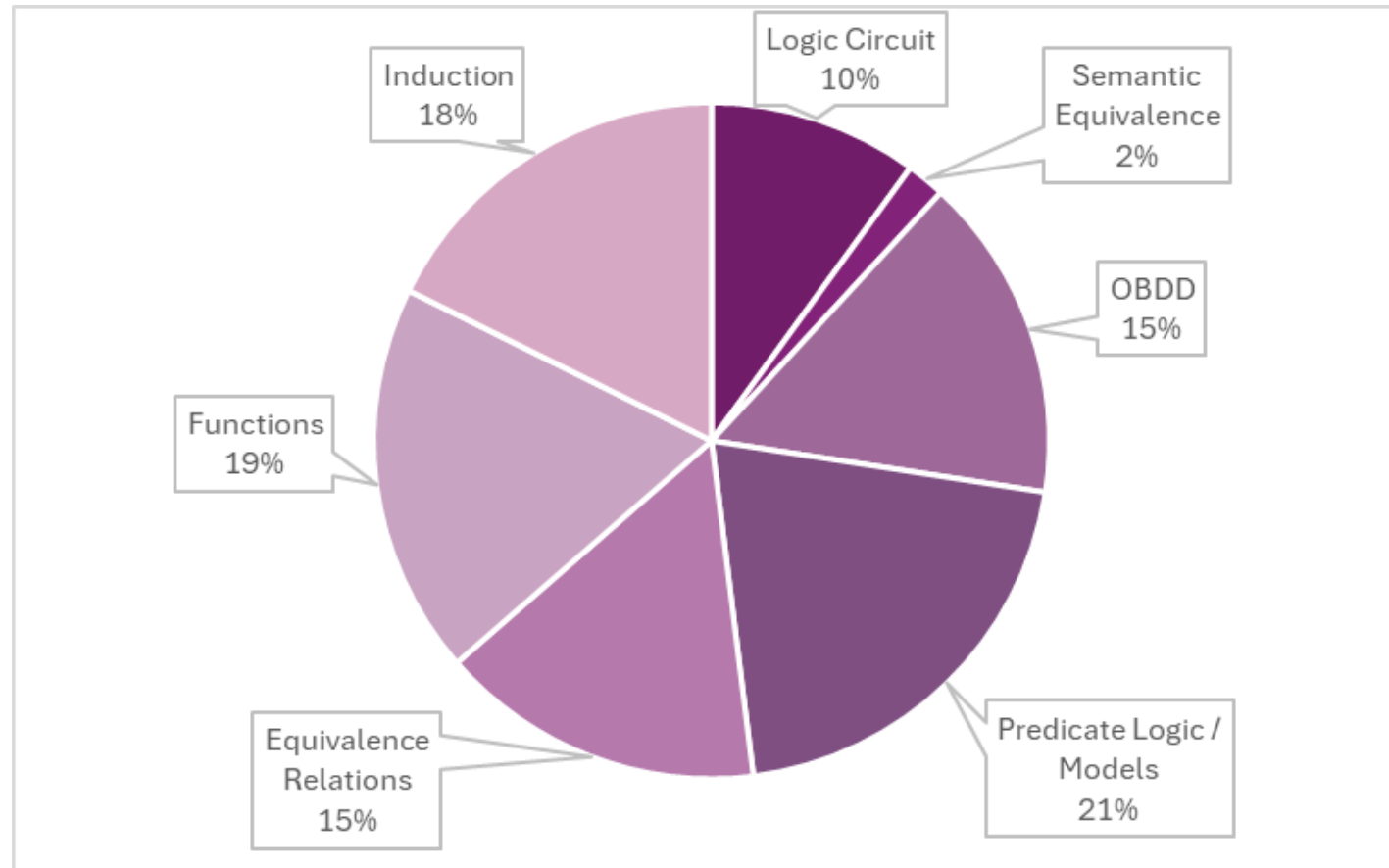
- Bachelor's degree Artificial Intelligence
- Experience with teaching for Athena
- Currently enjoying my gap year: travelling abroad!

WhatsApp



Exam Analysis

Final Exams 2019-2023



Planning

10-15 min break
every 45 min!

Wednesday	Logic week 5: ○ Logic Circuits Logic week 6: ○ OBDDs Logic week 7: ○ Predicate Logic ○ Models
Thursday	Sets week 5 + 6 + 7

Pro-active participation

- Raise your hand / use the chat / unmute to ask questions!
- Be interactive!
- Take notes!
- It is okay to make mistakes 😊
- Slides will be sent afterwards in the whatsapp group



Structure



Structure based on past exams



Learn the most by asking questions

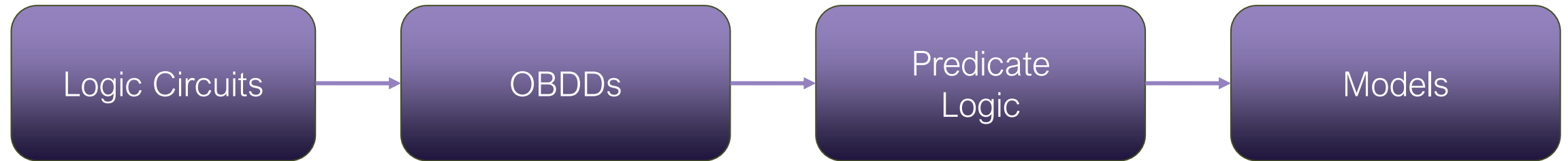


Active learning with old exam questions is the basis of our trainings



24/7 WhatsApp aid

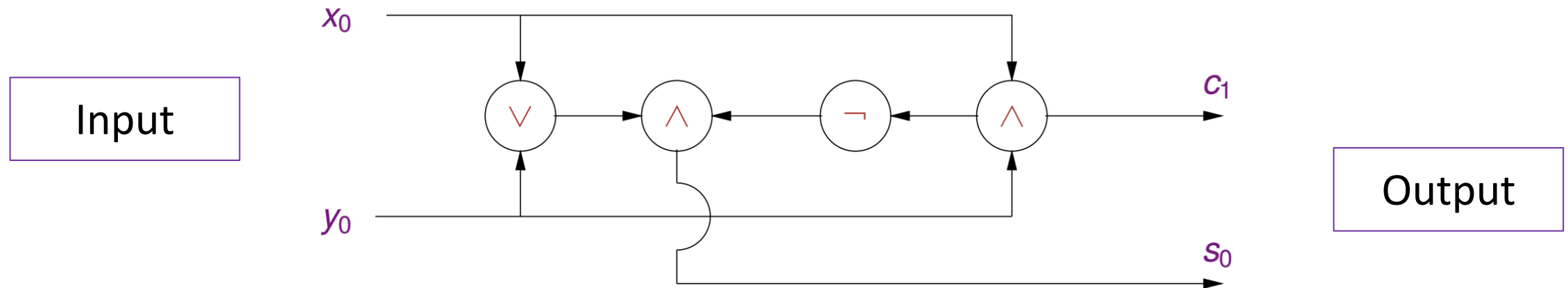
Planning Day 1



Logic Circuits

Logic Circuits

A logic circuit is a **representation** of a boolean function



A logic circuit is a **representation** of a boolean function

Definition

A *Boolean function* maps

- ▶ tuples (e.g. pairs, triples) of the truth values 1 and 0
- ▶ to the truth values 1 and 0.

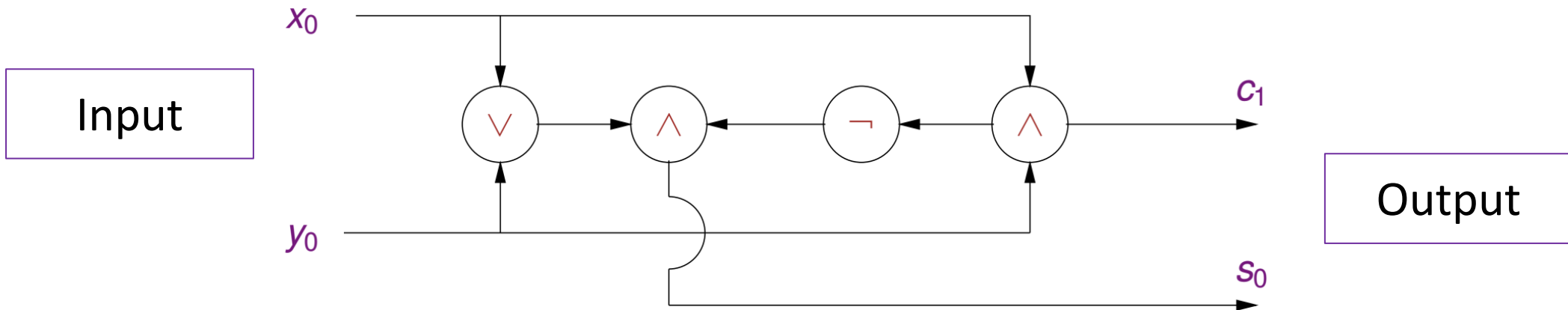
Logic Circuits

A logic circuit is a **representation** of a boolean function

Definition

A *Boolean function* maps

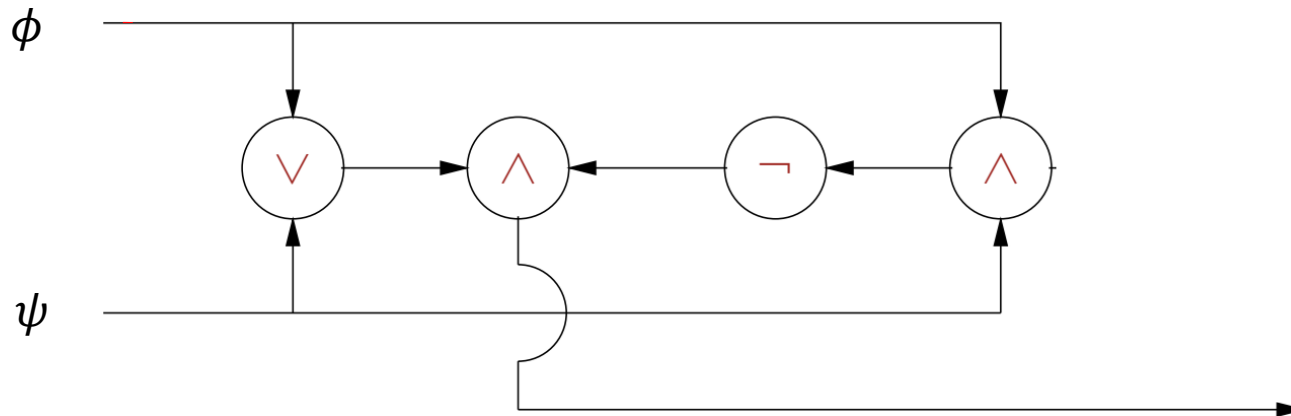
- ▶ tuples (e.g. pairs, triples) of the truth values **1** and **0**
- ▶ to the truth values **1** and **0**.



Logic Circuits: Example

What propositional logic formula corresponds to the logic circuit?

Reads similar to a
parse tree!

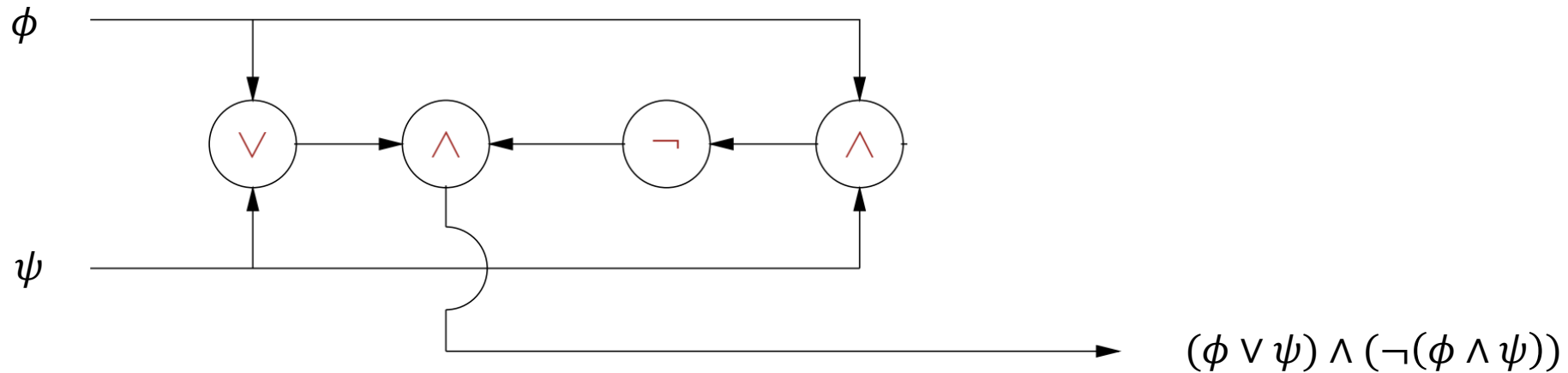


$$(\phi \vee \psi) \wedge (\neg (\phi \wedge \psi))$$

Logic Circuits: Example

What propositional logic formula corresponds to the logic circuit?

Reads similar to a
parse tree!



Logic Circuits: Example!

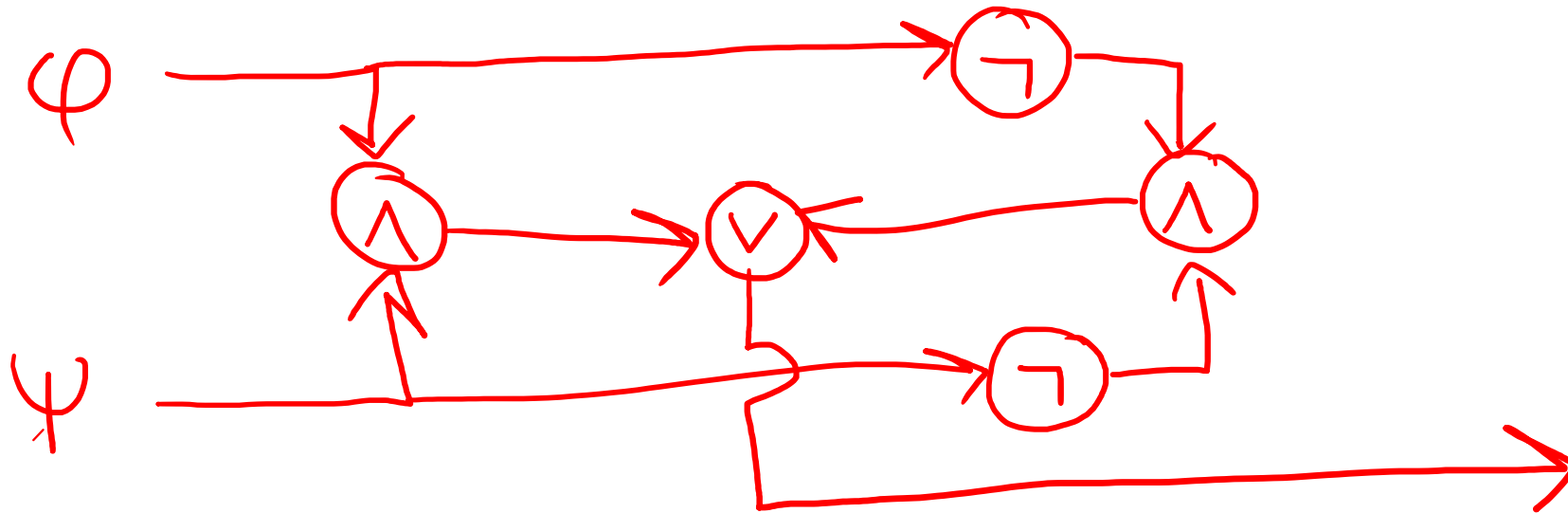
Exam Question: Final 2020

5. Logic circuits (12 points)

Give a logic circuit corresponding to the formula $\phi \leftrightarrow \psi$, using **only AND, OR and NOT gates**.

$$\phi \leftrightarrow \psi, (\phi \wedge \psi) \vee (\neg \phi \wedge \neg \psi)$$

ϕ	ψ	$\phi \leftrightarrow \psi$
T	T	T
F	F	T
T	F	F
F	T	F



Logic Circuits: Example!

Exam Question: Final 2020

5. Logic circuits (12 points)

Give a logic circuit corresponding to the formula $\phi \leftrightarrow \psi$, using only AND, OR and NOT gates.

1. Rewrite the formula, such that it only uses $\wedge, \vee, \neg$
$$\phi \leftrightarrow \psi \equiv (\phi \wedge \psi) \vee (\neg \phi \wedge \neg \psi)$$

Logic Circuits: Example!

Exam Question: Final 2020

5. Logic circuits (12 points)

Give a logic circuit corresponding to the formula $\phi \leftrightarrow \psi$, using only AND, OR and NOT gates.

1. Rewrite the formula, such that it only uses $\wedge, \vee, \neg$
$$\phi \leftrightarrow \psi \equiv (\phi \wedge \psi) \vee (\neg \phi \wedge \neg \psi)$$
2. Identify the symbol with the highest priority, this is closest to the output!

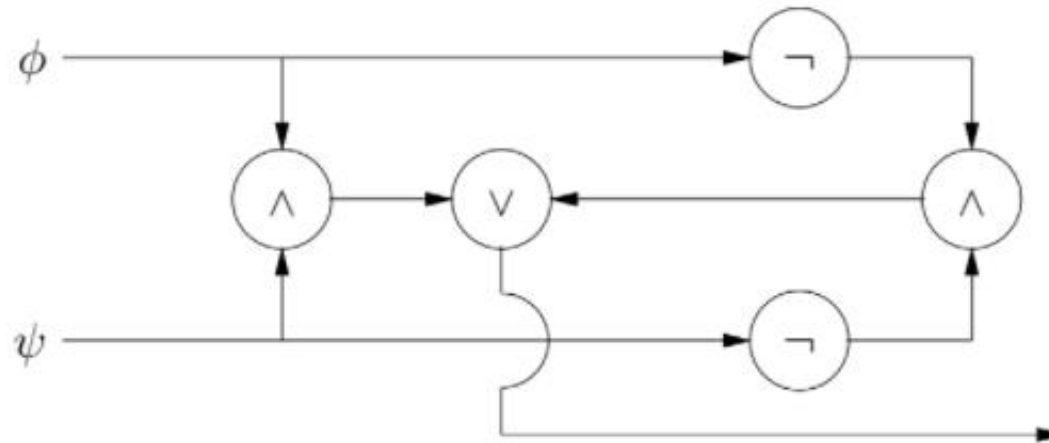
Logic Circuits: Example!

Exam Question: Final 2020

5. Logic circuits (12 points)

Give a logic circuit corresponding to the formula $\phi \leftrightarrow \psi$, using only AND, OR and NOT gates.

Solution: $\phi \leftrightarrow \psi$ is semantically equivalent to $(\phi \wedge \psi) \vee (\neg\phi \wedge \neg\psi)$. Hence a logic circuit for $\phi \leftrightarrow \psi$ is:



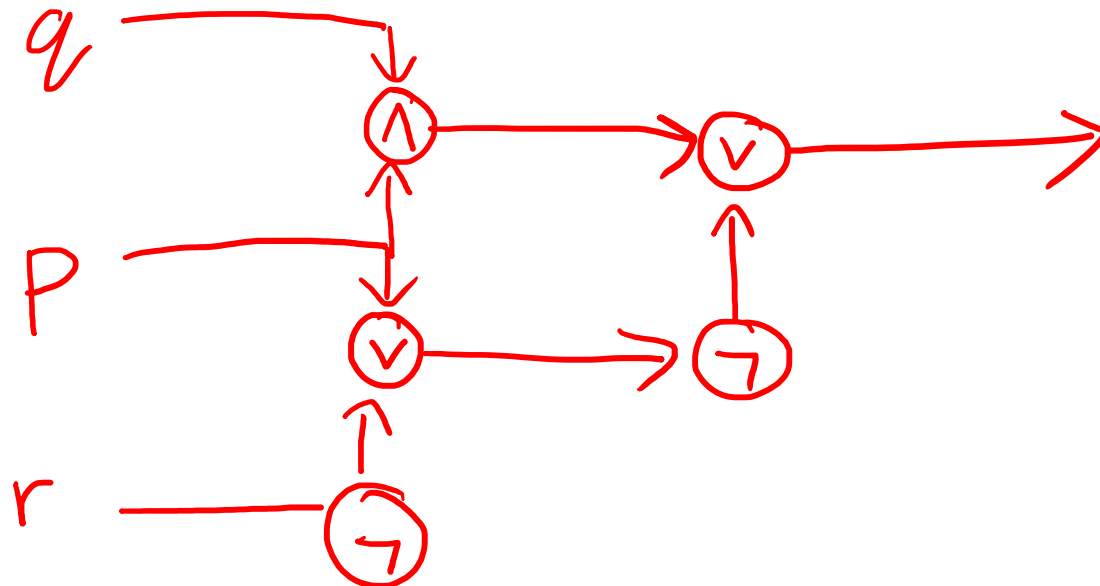
Logic Circuits: Exercise!

Exam Question: Final 2019

2. Logic circuit (7 points)

Construct a logic circuit that corresponds to the formula

$$(p \wedge q) \vee \neg(p \vee \neg r)$$



Logic Circuits: Exercise!

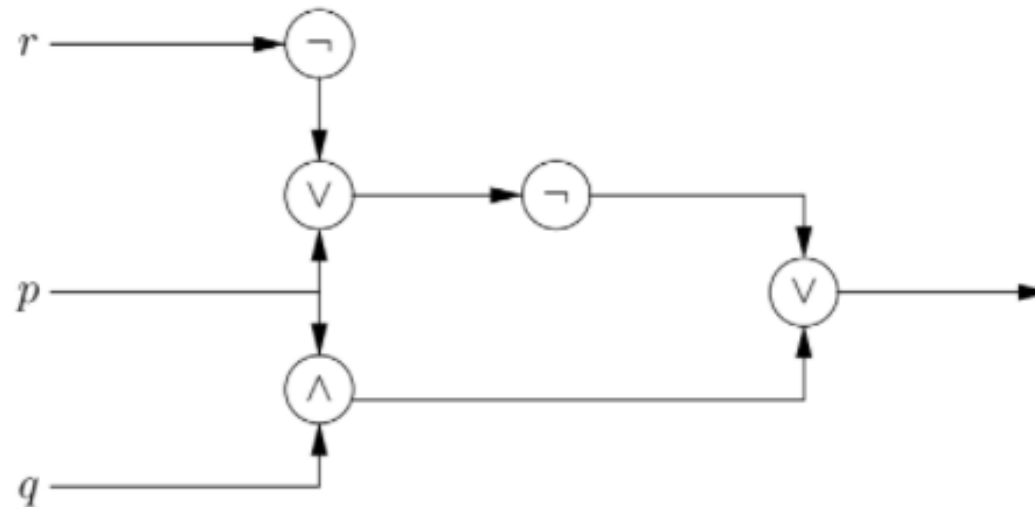
Exam Question: Final 2019

2. Logic circuit (7 points)

Construct a logic circuit that corresponds to the formula

$$(p \wedge q) \vee \neg(p \vee \neg r)$$

Solution:



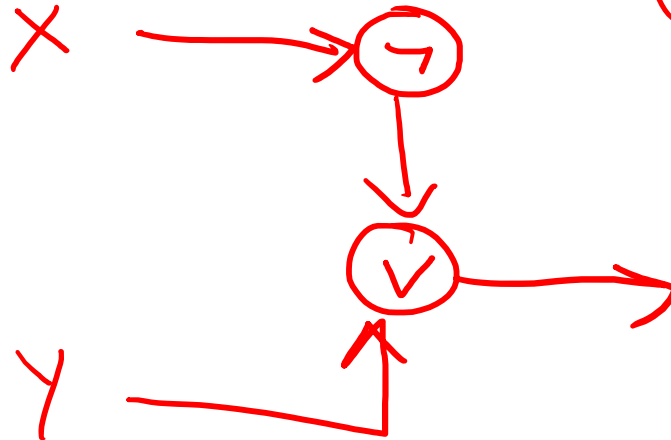
Logic Circuits: Exercise!

Exam Question: Final 2022

3. Logic circuit (10 points)

Give a logic circuit, using only AND, OR and NOT gates, that represents $x \rightarrow y$.

$\neg x \vee y$:



(or $\neg(x \wedge \neg y)$)

$$\begin{array}{r} x \rightarrow y \quad \triangle \\ \hline \neg x \vee y \quad \circ \end{array}$$

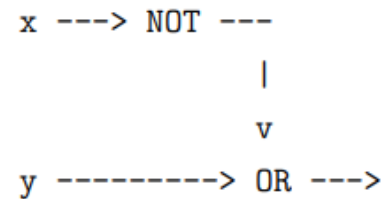
Logic Circuits: Exercise!

Exam Question: Final 2022

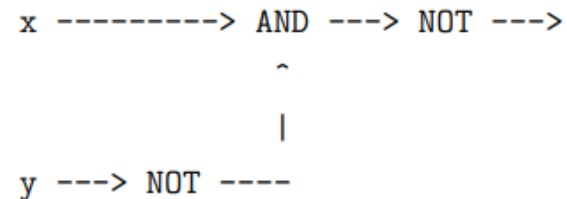
3. Logic circuit (10 points)

Give a logic circuit, using only AND, OR and NOT gates, that represents $x \rightarrow y$.

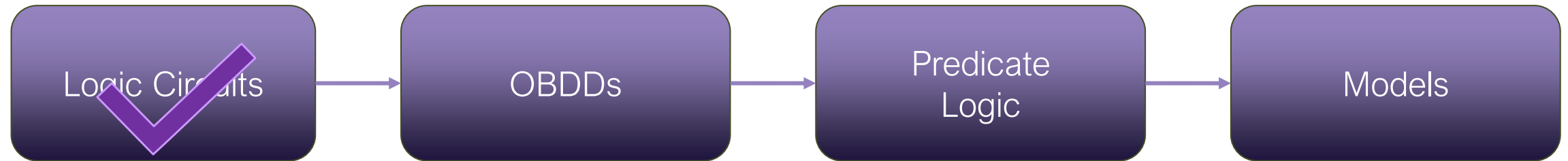
Solution: The easiest solution is to build a circuit for $\neg x \vee y$.



A slightly more complicated solution is to build a circuit for $\neg(x \wedge \neg y)$.



Planning Day 1



Break :)

Break time!

Meet back at

OBDDs

OBDDs

truth table

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Boolean function

$f(x, y)$
$f(1, 1) = 1$
$f(1, 0) = 0$
$f(0, 1) = 1$
$f(0, 0) = 1$

Each truth table expresses a *Boolean function*

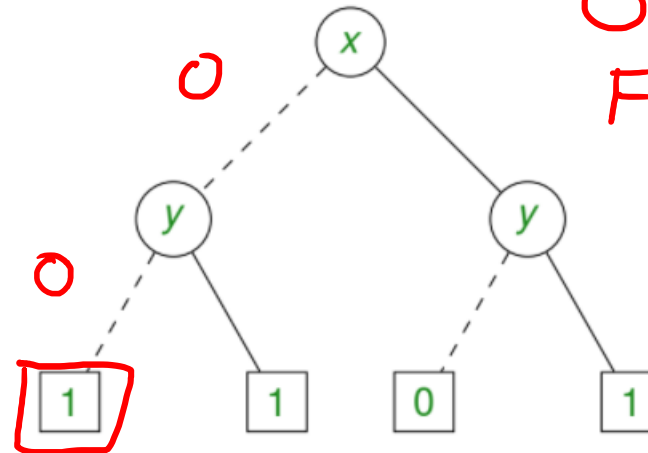
- ▶ from tuples (e.g. pairs, triples) of the truth values 1 and 0
- ▶ to the truth values 1 and 0.

Propositional variables p, q are now called *Boolean* variables x, y .

OBDDs

A Boolean function can be represented as a *binary decision tree*, with regard to some (total) order on its variables.

For example, for $x \rightarrow y$, with x placed above y :



$$\begin{aligned} 0 \rightarrow 0 &= 1 \\ F \rightarrow F &= T \end{aligned}$$

A *dashed* line (to the left) corresponds to the value 0, and a *solid* line (to the right) to the value 1, for the variable above.

Non-leaves are *circles*; they contain a *Boolean variable*.

Leaves are *boxes*; they contain 0 or 1.

OBDDs

An *ordered binary decision diagram (OBDD)* is obtained by **collapsing** and **eliminating** nodes in a binary decision tree.

C1 All leaves labeled **0** are **collapsed**, and also all leaves labeled **1**.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

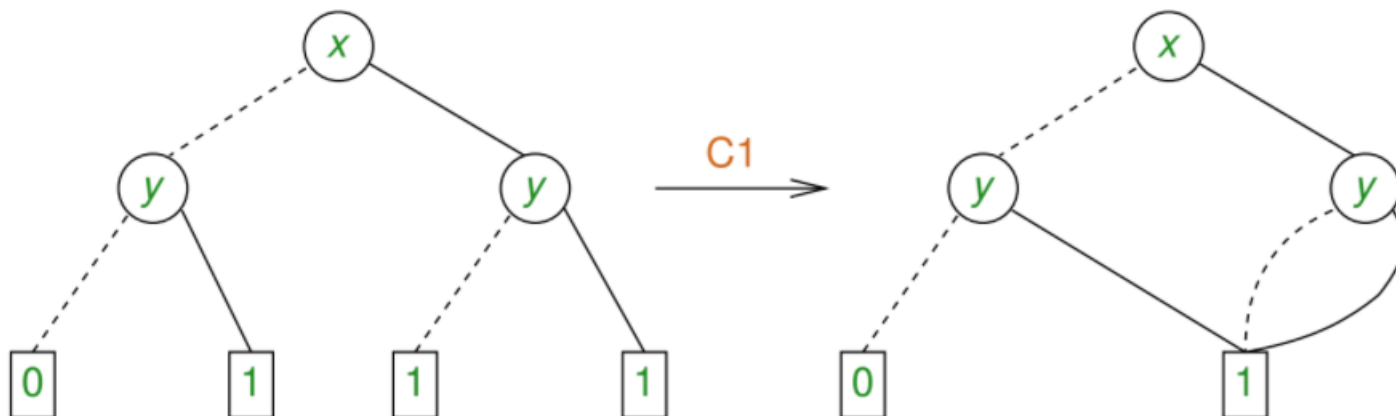
C2 If the **0**- and **1**-edge of non-leaf n point to the *same* node n' ,
* then n is **eliminated**, and
* its incoming edges are **redirected** to n' .

C3 (Two or more) non-leaves are **collapsed** to one node if:

- they are associated with the same variable, and
- their **0**-edges point to the same node, and their **1**-edges point to the same node.

Rule **C1**: **Collapse identical leaves.**

Example: Consider the binary decision tree for $x \vee y$.

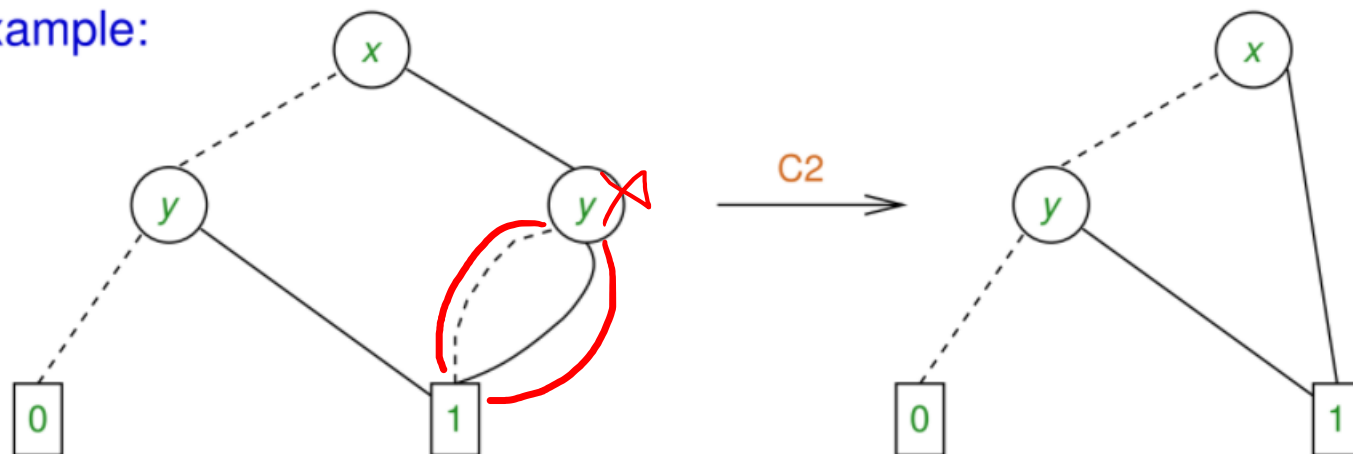


Rule **C2**: **Eliminate superfluous non-leaves.**

If both outgoing edges of a node n point to the same node n' , then:

- ▶ eliminate n , and
- ▶ redirect its incoming edges to n' .

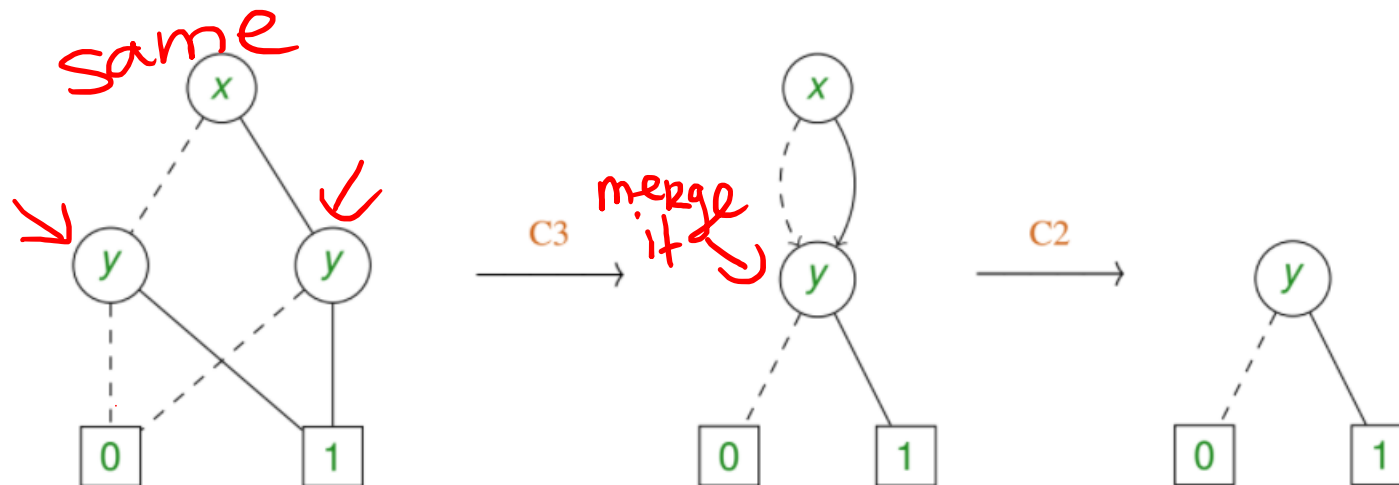
Example:



Rule **C3**: **Remove duplicate non-leaves.**

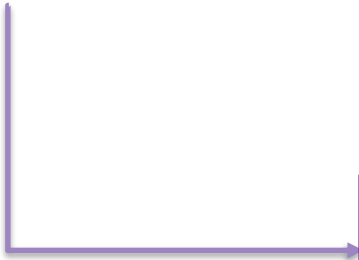
Collapse non-leaves associated with the same variable if their 0-edges and their 1-edges point to the same nodes.

Example: Consider the binary decision tree for $(x \wedge y) \vee (\neg x \wedge y)$, after collapsing its leaves with **C1**.



Definition

An OBDD is called *reduced* if none of the rules C1, C2 and C3 can be applied to it.



So apply the rules until they can no longer be applied to get a reduced OBDD!

Exam Question: Resit 2021

3. OBDD (12 points)

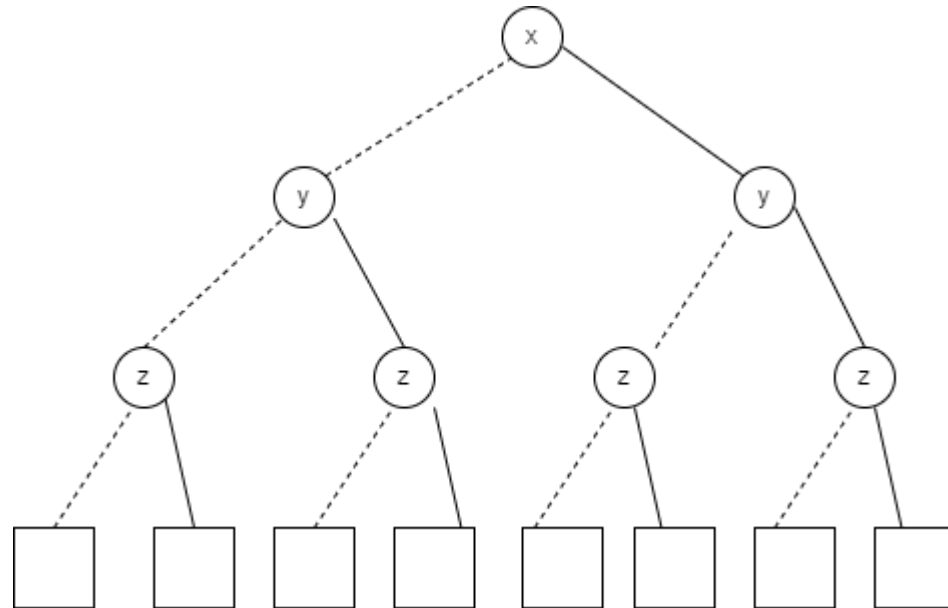
Represent $z \rightarrow (x \oplus y)$ by means of a binary decision tree, with respect to the variable ordering x, y, z . Moreover, reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)

OBDDs: Example

Exam Question: Resit 2021

3. OBDD (12 points)

Represent $z \rightarrow (x \oplus y)$ by means of a binary decision tree, with respect to the variable ordering x, y, z . Moreover, reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)

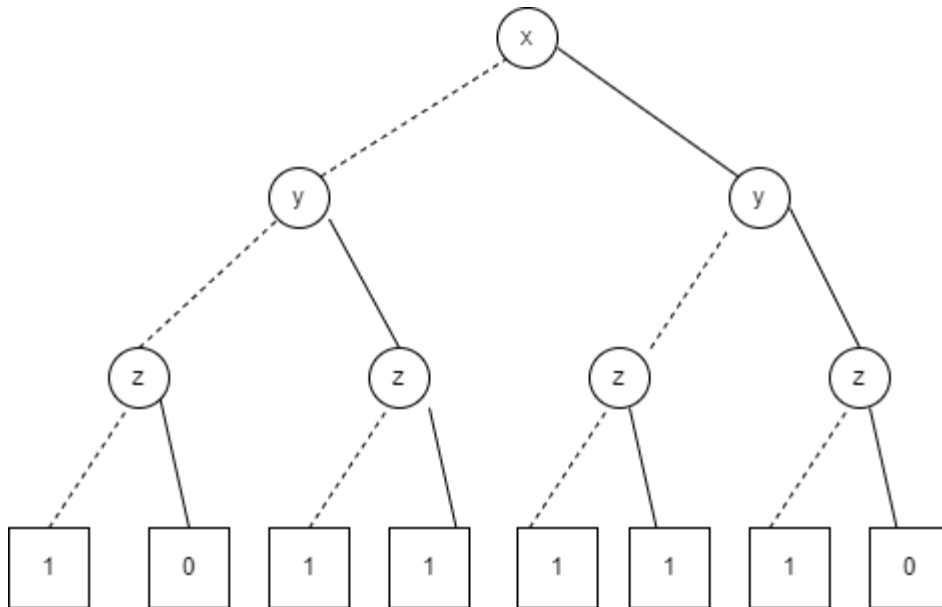


OBDDs: Example

Exam Question: Resit 2021

3. OBDD (12 points)

Represent $z \rightarrow (x \oplus y)$ by means of a binary decision tree, with respect to the variable ordering x, y, z . Moreover, reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)



OBDDs: Example

Exam Question: Resit 2021

3. OBDD (12 points)

Represent $z \rightarrow (x \oplus y)$ by means of a binary decision tree, with respect to the variable ordering x, y, z . Moreover, reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)

C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

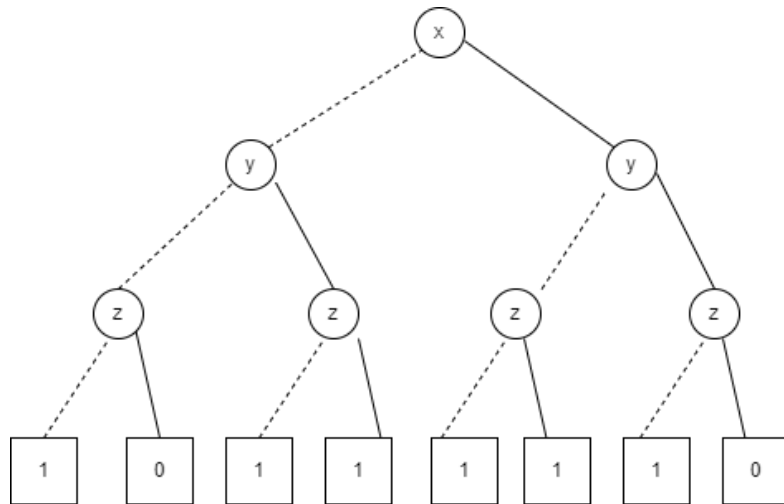
Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

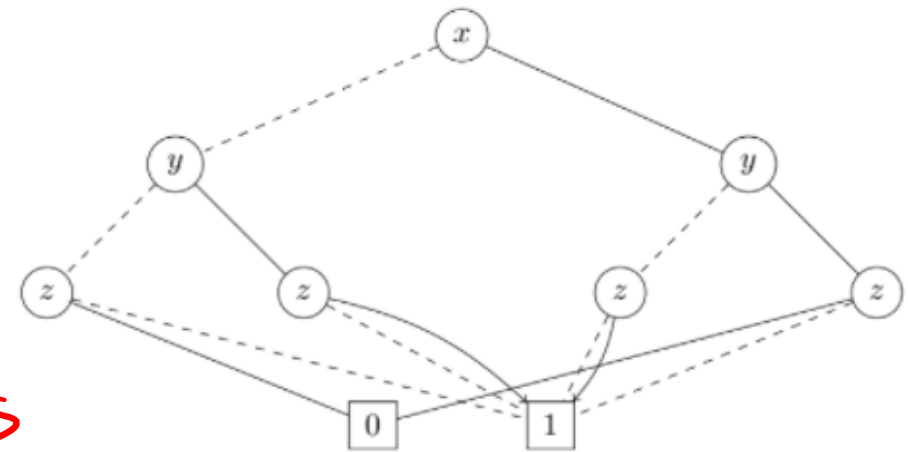
C3 (Two or more) non-leaves are collapsed to one node if:

- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.



Apply C1 to collapse the 0- and the 1-leaves.

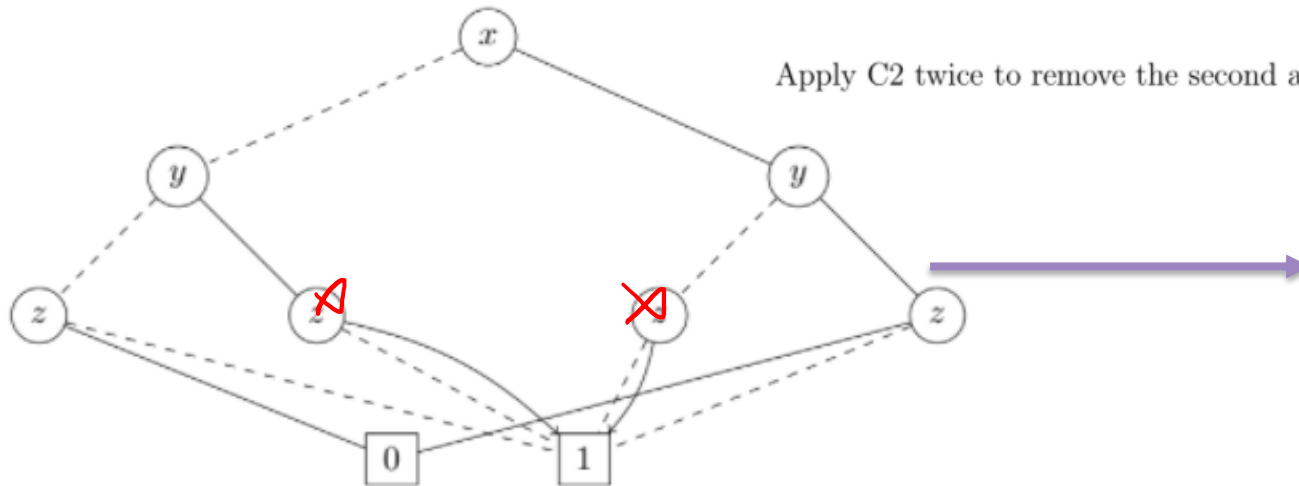
collapse
n nodes



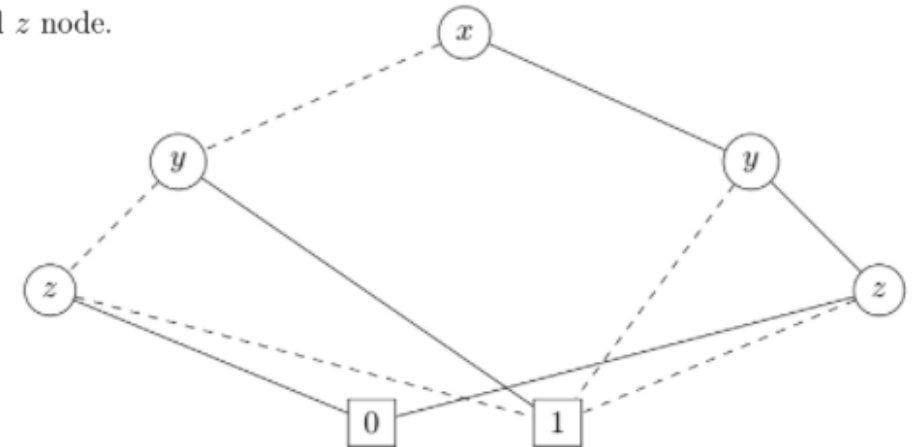
Exam Question: Resit 2021

3. OBDD (12 points)

Represent $z \rightarrow (x \oplus y)$ by means of a binary decision tree, with respect to the variable ordering x, y, z . Moreover, reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)



Apply C2 twice to remove the second and the third z node.



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

C3 (Two or more) non-leaves are collapsed to one node if:

- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

OBDDs: Example

Exam Question: Resit 2021

3. OBDD (12 points)

Represent $z \rightarrow (x \oplus y)$ by means of a binary decision tree, with respect to the variable ordering x, y, z . Moreover, reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)

C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

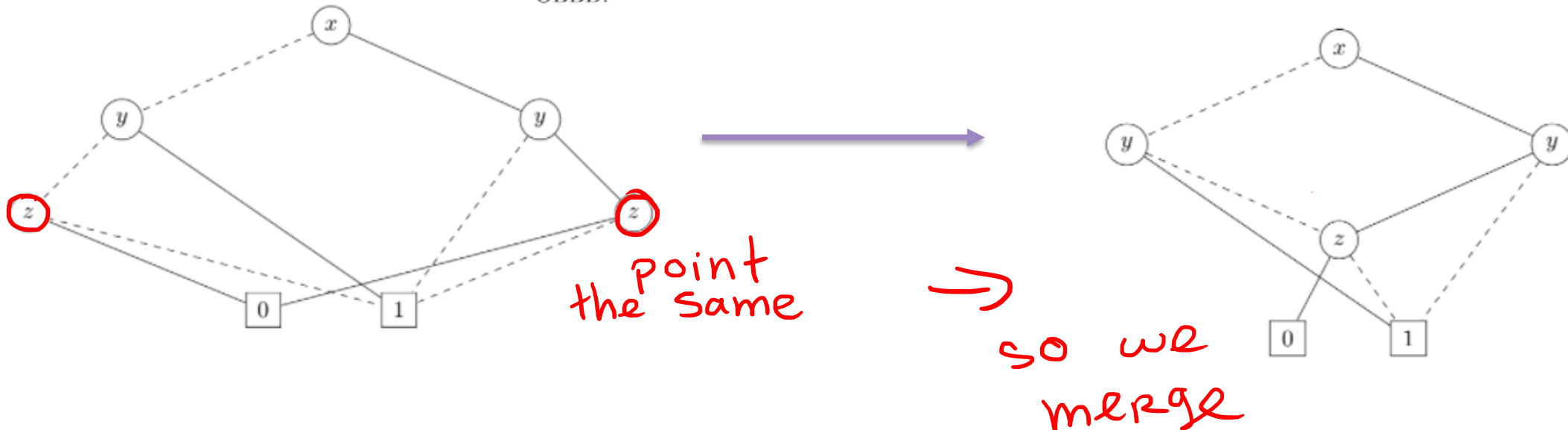
C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is **eliminated**, and
- * its incoming edges are **redirected** to n' .

C3 (Two or more) non-leaves are **collapsed** to one node if:

- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

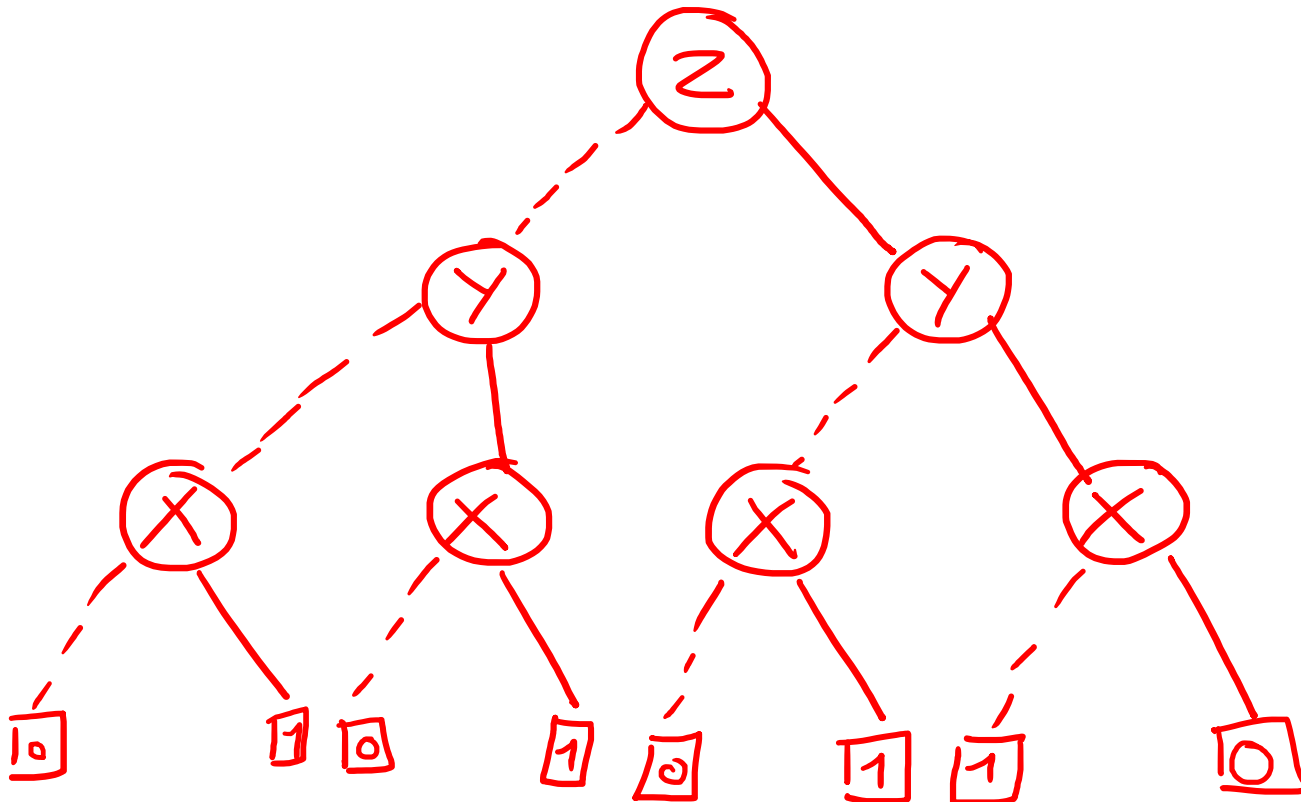
Finally, apply C3 to collapse the two remaining z nodes. This results in the reduced OBDD.



Exam Question: Resit 2021

5. OBDDs (4+10 points)

- (a) Represent $x \oplus (y \wedge z)$ by means of a binary decision tree, with respect to the variable ordering z, y, x .



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1 .

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0 - and 1 -edge of non-leaf n point to the same node n' ,

* then n is eliminated, and

* its incoming edges are redirected to n' .

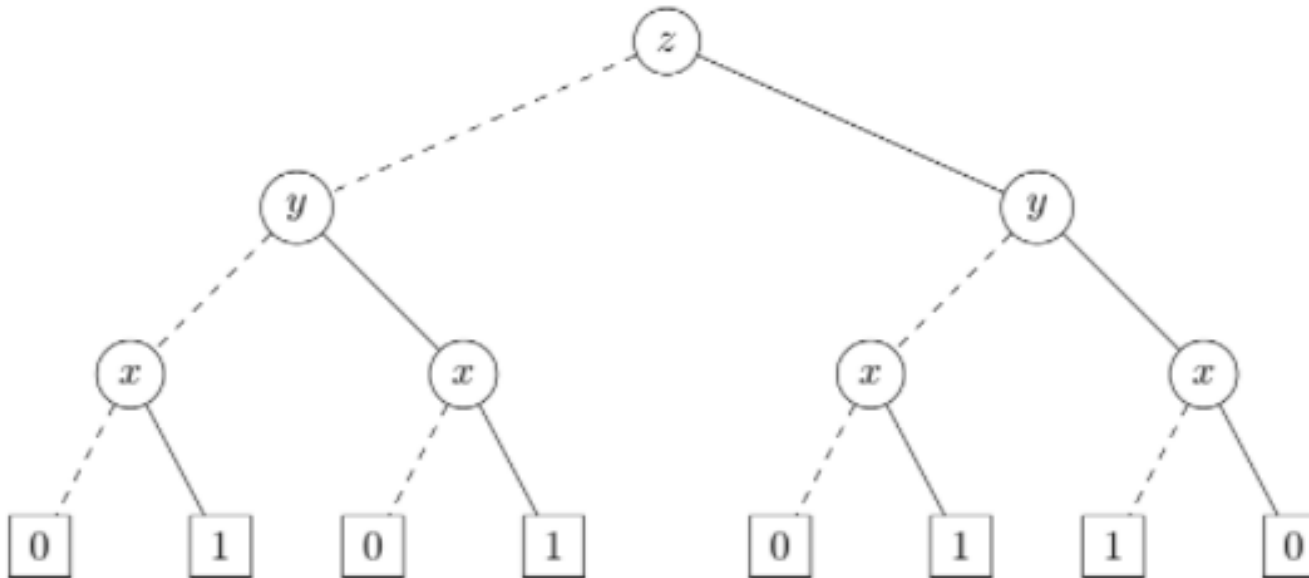
C3 (Two or more) non-leaves are collapsed to one node if:

- they are associated with the same variable, and
- their 0 -edges point to the same node, and their 1 -edges point to the same node.

Exam Question: Resit 2021

5. OBDDs (4+10 points)

- (a) Represent $x \oplus (y \wedge z)$ by means of a binary decision tree, with respect to the variable ordering z, y, x .



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

C3 (Two or more) non-leaves are collapsed to one node if:

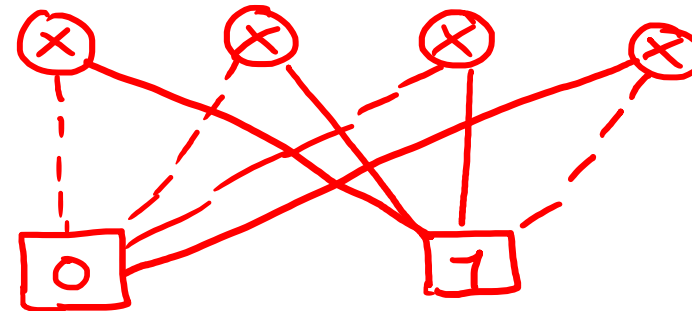
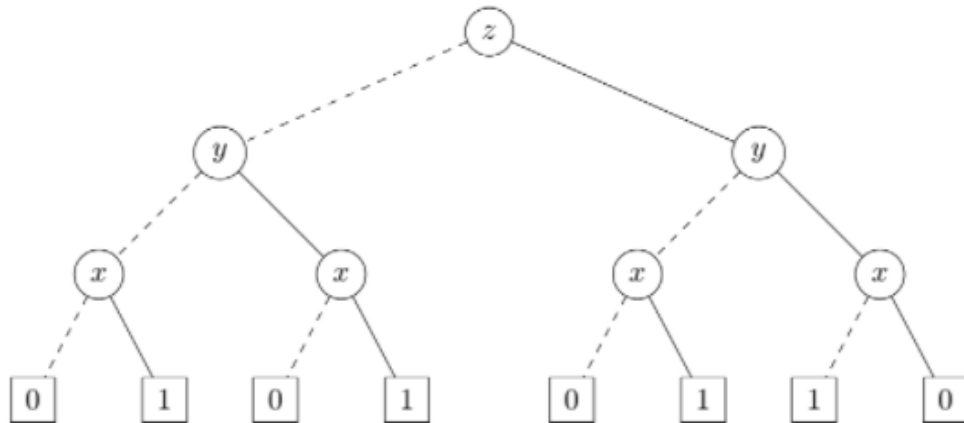
- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

OBDDs: Exercise!

Exam Question: Resit 2021

5. OBDDs (4+10 points)

- (a) ~~Represent $x \oplus (y \wedge z)$ by means of a binary decision tree, with respect to the variable ordering z, y, x .~~
- (b) Reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

C3 (Two or more) non-leaves are collapsed to one node if:

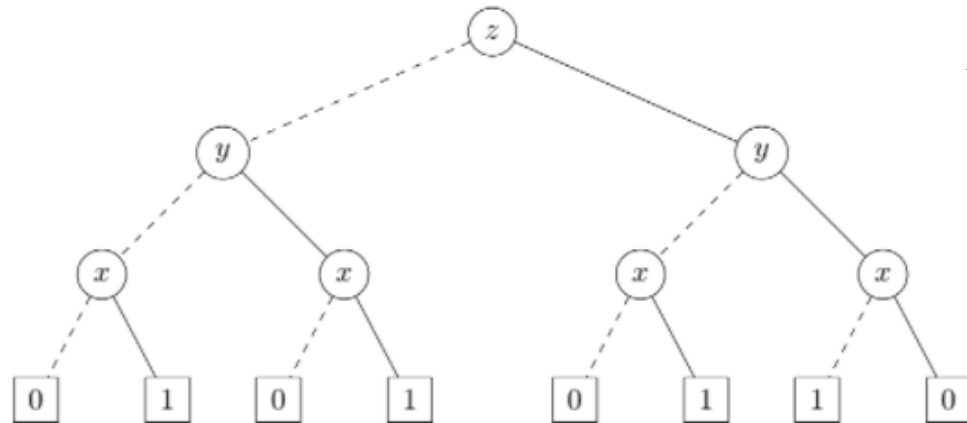
- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

OBDDs: Exercise!

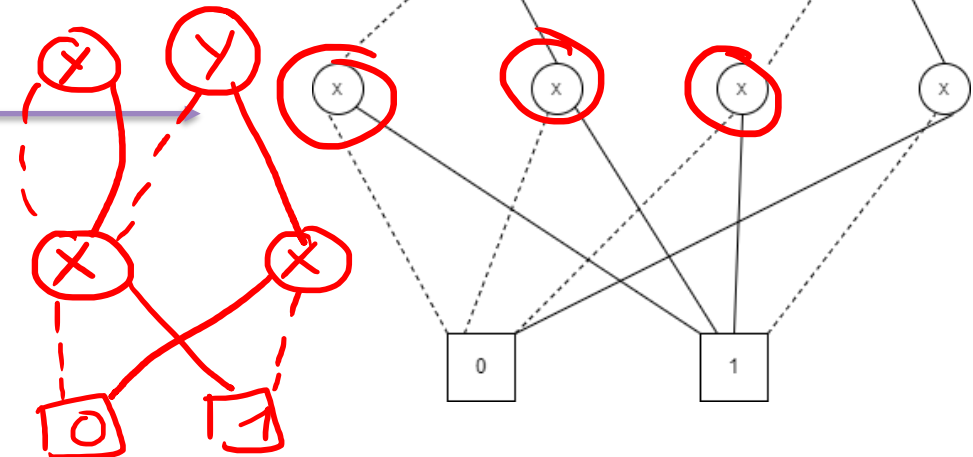
Exam Question: Resit 2021

5. OBDDs (4+10 points)

- (a) ~~Represent $x \oplus (y \wedge z)$ by means of a binary decision tree, with respect to the variable ordering z, y, x .~~
- (b) Reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)



Apply c1 to collapse leaves



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

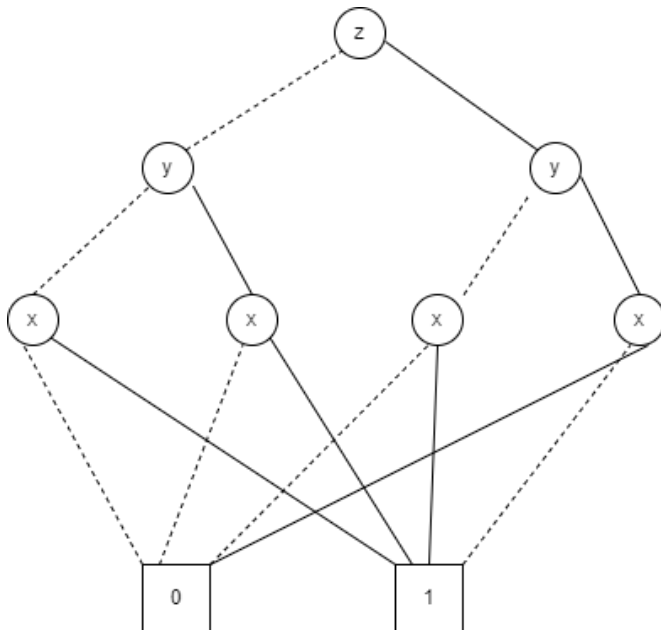
C3 (Two or more) non-leaves are collapsed to one node if:

- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

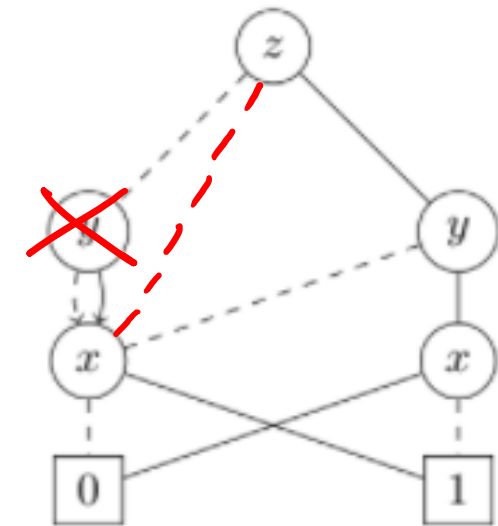
Exam Question: Resit 2021

5. OBDDs (4+10 points)

- (a) ~~Represent $x \oplus (y \wedge z)$ by means of a binary decision tree, with respect to the variable ordering z, y, x .~~
- (b) Reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)



Apply c3 to 3 x's



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

C3 (Two or more) non-leaves are collapsed to one node if:

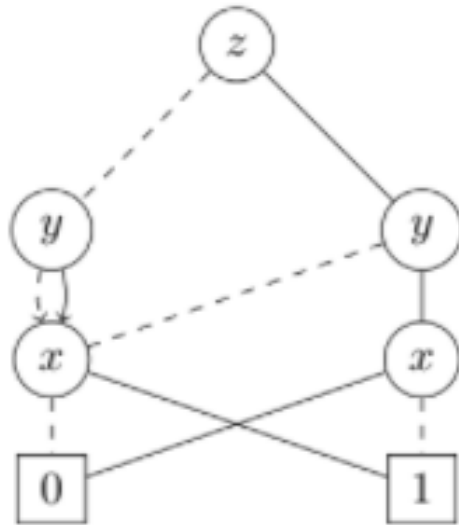
- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

OBDDs: Exercise!

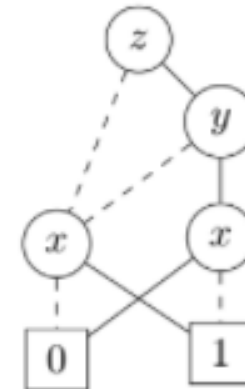
Exam Question: Resit 2021

5. OBDDs (4+10 points)

- (a) ~~Represent $x \oplus (y \wedge z)$ by means of a binary decision tree, with respect to the variable ordering z, y, x .~~
- (b) Reduce this binary decision tree to an ordered binary decision diagram. (Also give the intermediate reduction steps.)



Apply C2 to remove the left-hand node y :



C1 All leaves labeled 0 are collapsed, and also all leaves labeled 1.

Next, two reduction steps are performed repeatedly, until neither of them can be applied anymore:

C2 If the 0- and 1-edge of non-leaf n point to the same node n' ,

- * then n is eliminated, and
- * its incoming edges are redirected to n' .

C3 (Two or more) non-leaves are collapsed to one node if:

- they are associated with the same variable, and
- their 0-edges point to the same node, and their 1-edges point to the same node.

Existential and Universal Quantifiers:

Existential: $\exists x$ (there exists an x , such that...)

Universal: $\forall x$ (for all x , ...)

Existential and Universal Quantifiers:

Existential: $\exists x$ (there exists an x , such that...)

Universal: $\forall x$ (for all x , ...)

$\exists x: 2x = 4 \rightarrow$ There is some x for which the equation $2x = 4$ holds

Existential and Universal Quantifiers:

Existential: $\exists x$ (there exists an x , such that...)

Universal: $\forall x$ (for all x , ...)

$\exists x: 2x = 4 \rightarrow$ There is some x for which the equation $2x = 4$ holds

$\forall x: 2x = 2x \rightarrow$ For all x , the equation $2x = 2x$ holds

Existential and Universal Quantifiers:

Existential: $\exists x$ (there exists an x , such that...)

Universal: $\forall x$ (for all x , ...)

Let x be a Boolean variable and ϕ a logic formula.

$\forall x \phi$ is 1 (i.e., true) if:

- ▶ ϕ with 1 substituted for x yields 1, and
- ▶ ϕ with 0 substituted for x yields 1.

In other words, ϕ is true for *all* values of x .

Existential and Universal Quantifiers:

Existential: $\exists x$ (there exists an x , such that...)

Universal: $\forall x$ (for all x , ...)

Let x be a Boolean variable and ϕ a logic formula.

$\forall x \phi$ is 1 (i.e., true) if:

- ▶ ϕ with 1 substituted for x yields 1, and
- ▶ ϕ with 0 substituted for x yields 1.

In other words, ϕ is true for all values of x .

$\exists x \phi$ is 1 if:

- ▶ ϕ with 1 substituted for x yields 1, or
- ▶ ϕ with 0 substituted for x yields 1.

In other words, ϕ is true for some value of x .

OBDDs

Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1 -child of this node.

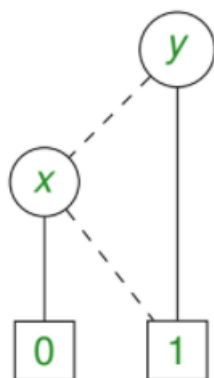
We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0 -child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

OBDDs: Example

Example: We build an OBDD for $\forall x (x \rightarrow y)$, against the order y, x .



$x \rightarrow y$

Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1 -child of this node.

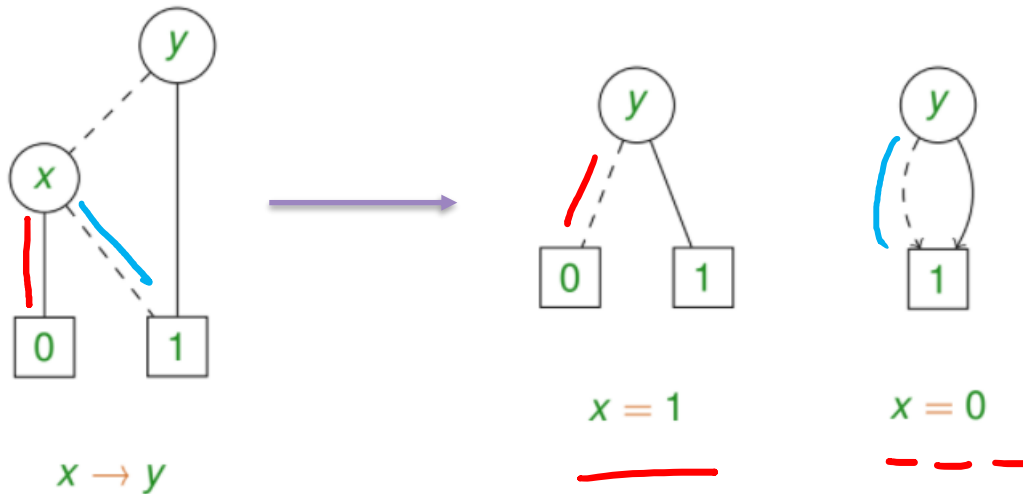
We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0 -child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

OBDDs: Example

Example: We build an OBDD for $\forall x (x \rightarrow y)$, against the order y, x .



Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1 -child of this node.

We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0 -child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

OBDDs: Example

Example: We build an OBDD for $\forall x (x \rightarrow y)$, against the order y, x .



$$0 \wedge 1 = 0$$

$$1 \wedge 1 = 1$$

Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- Eliminate all nodes from O that are associated to x .
- Redirect each incoming edge of an eliminated node to the 1 -child of this node.

We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- Eliminate all nodes from O that are associated to x .
- Redirect each incoming edge of an eliminated node to the 0 -child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

$\exists x O$ is captured by the OBDD for $O(1) \vee O(0)$.

Logical operations on OBDDs

Given OBDDs O_1 and O_2 , *with regard to the same variable order*.

We build an OBDD representing the *disjunction* of O_1 and O_2 .

Question

What to do if O_1 consists of a single leaf node ?

Logical operations on OBDDs *disjunction*

Given OBDDs O_1 and O_2 , with regard to the same variable order.

We build an OBDD representing the *disjunction* of O_1 and O_2 .

Question

What to do if O_1 consists of a single leaf node ?

Answer: If O_1 is the leaf 0, then return O_2 .

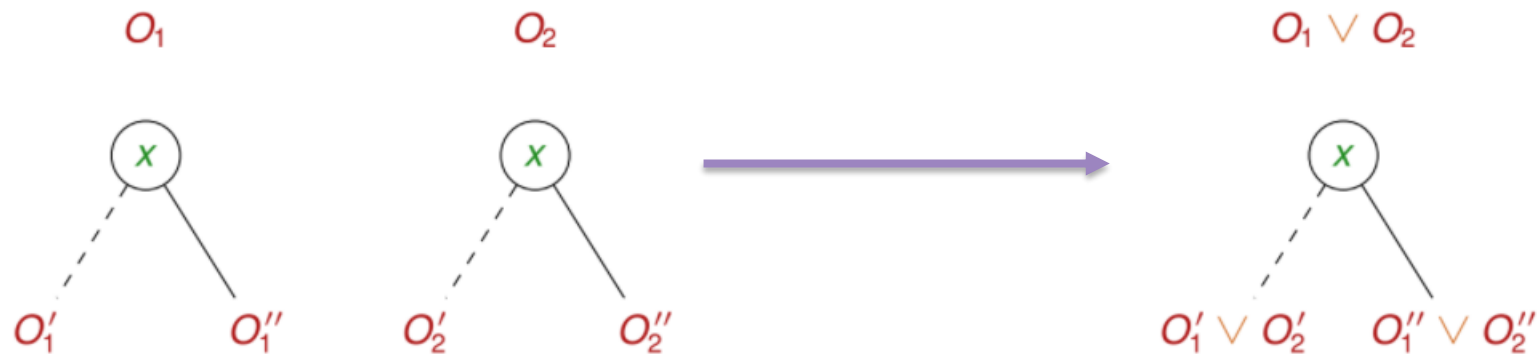
If O_1 is the leaf 1, then return the leaf 1.

Likewise if O_2 consists of a single leaf node.

Logical operations on OBDDs *disjunction*

Let O_1 and O_2 *not* consist of a single leaf node.

Case 1: The roots of O_1 and O_2 are associated to the same variable x .



Logical operations on OBDDs *disjunction*

Let O_1 and O_2 *not* consist of a single leaf node.

Case 2: The roots of the non-leaves O_1 and O_2 are associated to *different* variables x and y , respectively.

x comes before y in the variable order.



OBDDs: Exercise!

Exam Question: Final 2019

3. OBDD (2 + 7 + 7 points)

(a) Represent the formula

$$(\neg x \vee y) \wedge (x \vee \neg z)$$

by means of a binary decision tree, with respect to the variable ordering x, y, z .

(b) Reduce the binary decision tree from (a) to a reduced OBDD, using the steps C1,2,3.

(c) Construct an OBDD for

$$\forall y ((\neg x \vee y) \wedge (x \vee \neg z))$$

from the reduced OBDD you constructed in (b).

Try at home!

OBDDs: Exercise!

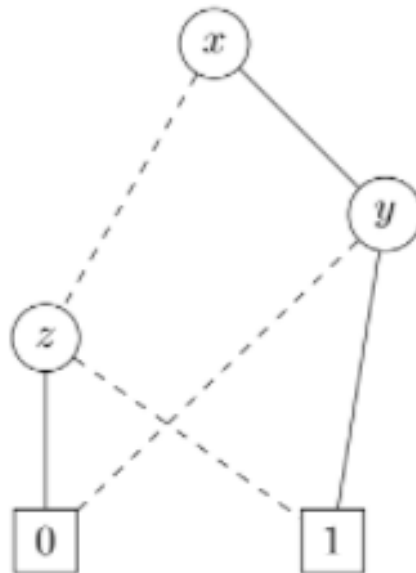
Exam Question: Final 2019

(c) Construct an OBDD for

$$\forall y ((\neg x \vee y) \wedge (x \vee \neg z))$$

from the reduced OBDD you constructed in (b).

Answer of b)



$$(\neg x \vee y) \wedge (x \vee \neg z)$$

Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1-child of this node.

We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0-child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

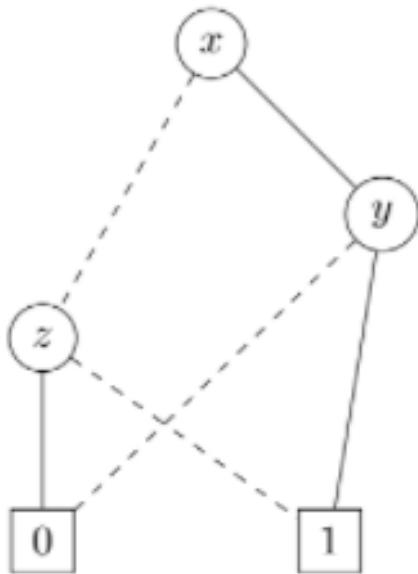
OBDDs: Exercise!

Exam Question: Final 2019

(c) Construct an OBDD for

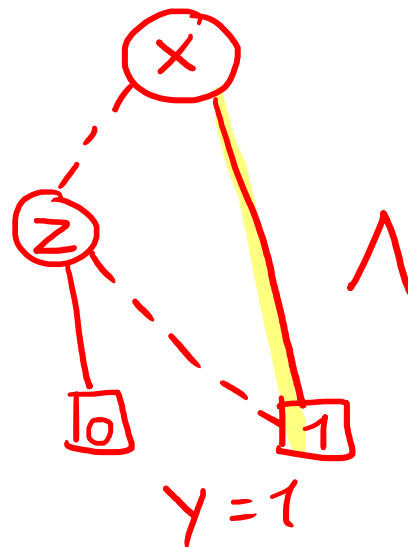
$$\forall y ((\neg x \vee y) \wedge (x \vee \neg z))$$

from the reduced OBDD you constructed in (b).



$$(\neg x \vee y) \wedge (x \vee \neg z)$$

$$\forall y \rightarrow O(0) \wedge O(1)$$



Given an OBDD O , we build an OBDD representing $\forall x O$.

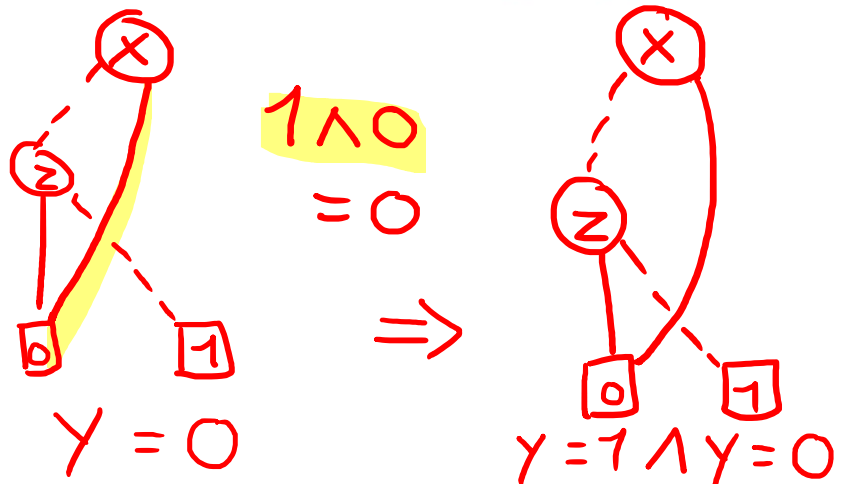
We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1-child of this node.

We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0-child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.



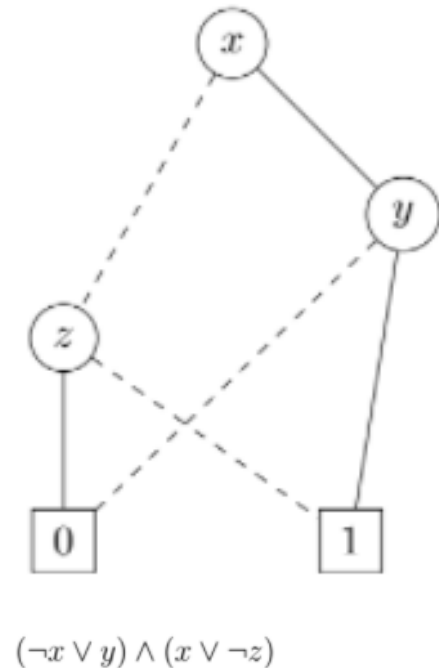
OBDDs: Exercise!

Exam Question: Final 2019

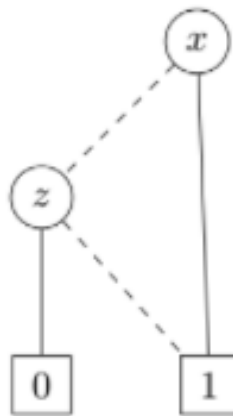
(c) Construct an OBDD for

$$\forall y ((\neg x \vee y) \wedge (x \vee \neg z))$$

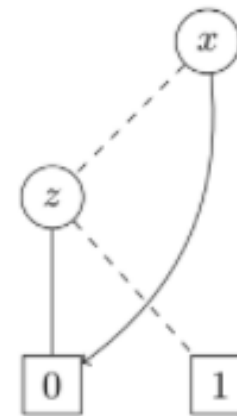
from the reduced OBDD you constructed in (b).



$$\forall y \rightarrow O(0) \wedge O(1)$$



$y = 1$



$y = 0$

Given an OBDD O , we build an OBDD representing $\forall x O$.

We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1-child of this node.

We construct an OBDD $O(0)$ representing O with 0 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0-child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.

OBDDs: Exercise!

Exam Question: Final 2019

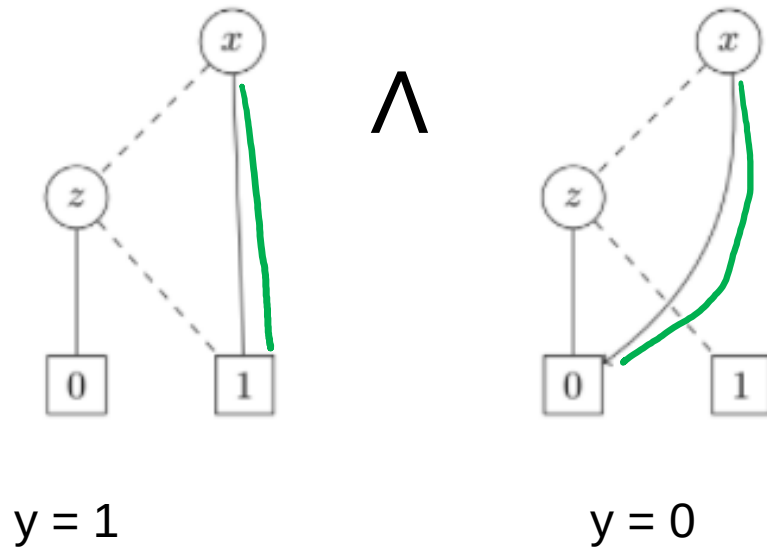
(c) Construct an OBDD for

$$\forall y ((\neg x \vee y) \wedge (x \vee \neg z))$$

from the reduced OBDD you constructed in (b).

$$\forall y \rightarrow O(0) \wedge O(1)$$

$$1 \wedge 0 = 0$$



Given an OBDD O , we build an OBDD representing $\forall x O$.

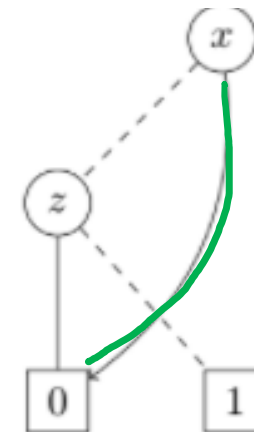
We construct an OBDD $O(1)$ representing O with 1 substituted for x :

- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 1 -child of this node.

We construct an OBDD $O(0)$ representing O with 0 substituted for x :

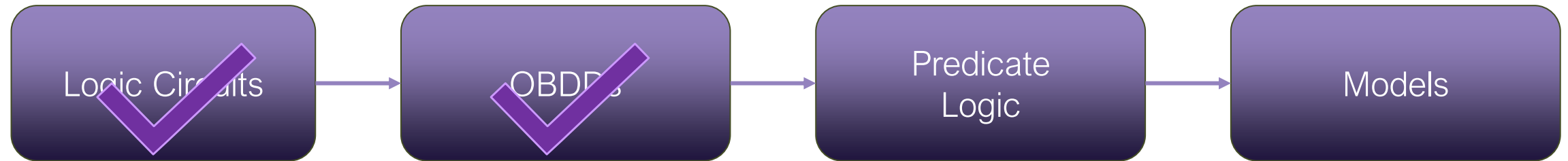
- ▶ Eliminate all nodes from O that are associated to x .
- ▶ Redirect each incoming edge of an eliminated node to the 0 -child of this node.

$\forall x O$ is captured by the OBDD for $O(1) \wedge O(0)$.



$$\exists x O \rightarrow O(1) \vee O(0)$$

Planning Day 1



Break :)

Break time!

Meet back at

11:17

Predicate Logic

Syntax of predicate logic

- ▶ atomic formulas, built from:
 - ▶ predicate symbols (like *Person*(_) or *Soulmate*(_,_))
 - ▶ constants (we will e.g. use *s* for Socrates)
 - ▶ variables (like *x* and *y*)
- ▶ connectives of propositional logic
- ▶ universal quantification $\forall x \phi$
- ▶ existential quantification $\exists x \phi$

Predicate Logic

Syntax of predicate logic

- ▶ atomic formulas, built from:
 - ▶ predicate symbols (like *Person*(_) or *Soulmate*(_,_))
 - ▶ constants (we will e.g. use *s* for Socrates)
 - ▶ variables (like *x* and *y*)
- ▶ connectives of propositional logic
- ▶ universal quantification $\forall x \phi$
- ▶ existential quantification $\exists x \phi$

- ▷ All human beings are mortal.
- ▷ Socrates is a human being.
- ▷ Therefore, Socrates is mortal.

Translation key

predicate *H*(*x*) : *x* is a human being

predicate *M*(*x*) : *x* is mortal

constant *s* : Socrates

Formalization:

$$\forall x (H(x) \rightarrow M(x)), H(s) \models M(s)$$

Predicate Logic

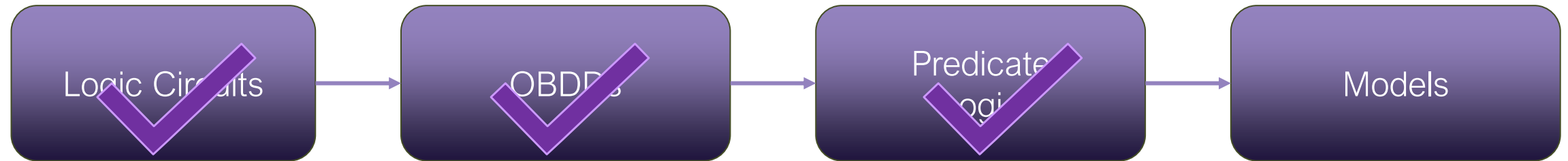
Examples

- ▶ $C(j) \wedge K(a, b)$ jane is clever and Anton knows betty
- ▶ $\neg K(a, a)$ anton does not know anton
- ▶ $K(a, b) \wedge \neg K(b, a)$ a knows b and b does not know A

Possible translation key

j : Jane	a : Anton	b : Betty
$C(x)$: x is clever	$K(x, y)$: x knows y	

Planning Day 1

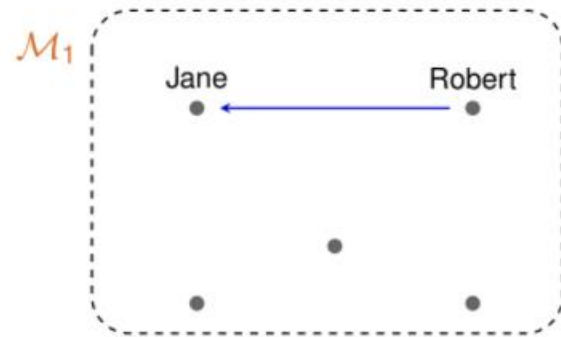


Models

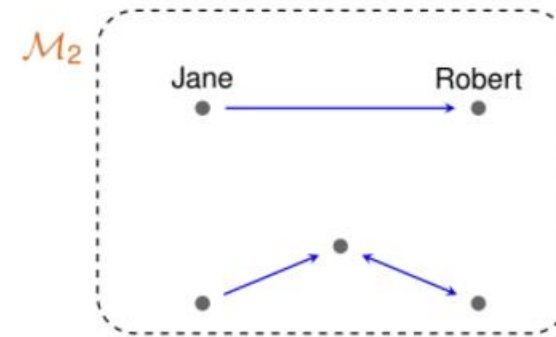
Models

$L(r, j)$ expresses that Robert loves Jane.

This relation is depicted below by blue arrows.



$\mathcal{M}_1 \models L(r, j)$



$\mathcal{M}_2 \not\models L(r, j)$

In model $\mathcal{M}_1$ (at the left) this holds.

But in model $\mathcal{M}_2$ (at the right) it does not.

Models

The *meaning* (or *truth value*) of a formula in predicate logic depends on the underlying **model** $\mathcal{M}$, consisting of:

- ▶ a non-empty **set** A of elements (**Jane**, **Robert**)
- ▶ an **interpretation**, with regard to elements in A , of the **constants** (r, j) and **predicate symbols** (L, C, K)

Models

The *meaning* (or *truth value*) of a formula in predicate logic depends on the underlying **model** $\mathcal{M}$, consisting of:

- ▶ a non-empty **set** A of elements (**Jane**, **Robert**)
- ▶ an **interpretation**, with regard to elements in A , of the **constants** (r, j) and **predicate symbols** (L, C, K)

$\forall x \phi$ holds in model $\mathcal{M}$ if ϕ holds for *every* element x in A :

Models

The *meaning* (or *truth value*) of a formula in predicate logic depends on the underlying **model** $\mathcal{M}$, consisting of:

- ▶ a non-empty **set** A of elements (**Jane**, **Robert**)
- ▶ an **interpretation**, with regard to elements in A , of the **constants** (r, j) and **predicate symbols** (L, C, K)

$\forall x \phi$ holds in model $\mathcal{M}$ if ϕ holds for every element x in A :

$\exists x \phi$ holds in model $\mathcal{M}$ if ϕ holds for some element x in A :

Models

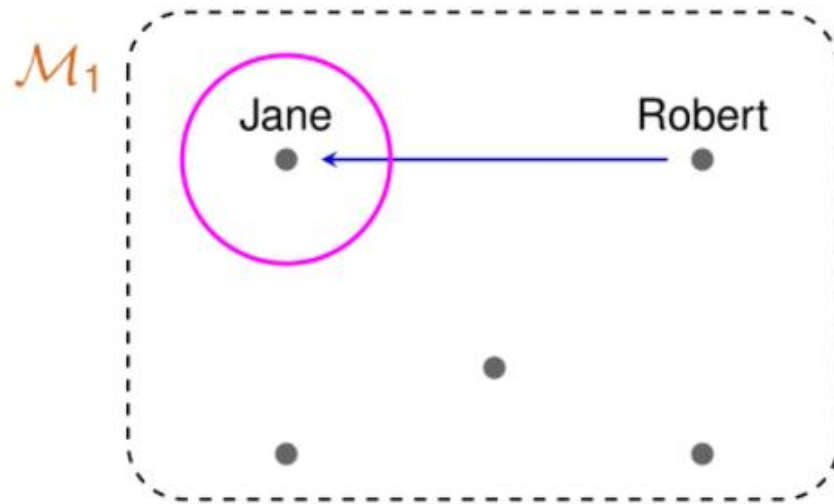
we use a *model* $\mathcal{M}$ consisting of:

- ▶ a **set** of elements; quantifiers range over this set
- ▶ an **interpretation** of the constants and predicate symbols

Example *model* $\mathcal{M}$

- ▶ A = all human beings (set)
- ▶ $j^{\mathcal{M}} = \text{Jane}$, $r^{\mathcal{M}} = \text{Robert}$ (elements)
- ▶ $C^{\mathcal{M}} = \{e \in A \mid e \text{ is clever}\}$ (subset)
- ▶ $L^{\mathcal{M}} = \{\langle e, f \rangle \mid e \in A, f \in A, e \text{ loves } f\}$ (relation)

Models



Here we are in a hypothetical world (model $\mathcal{M}_1$) where:

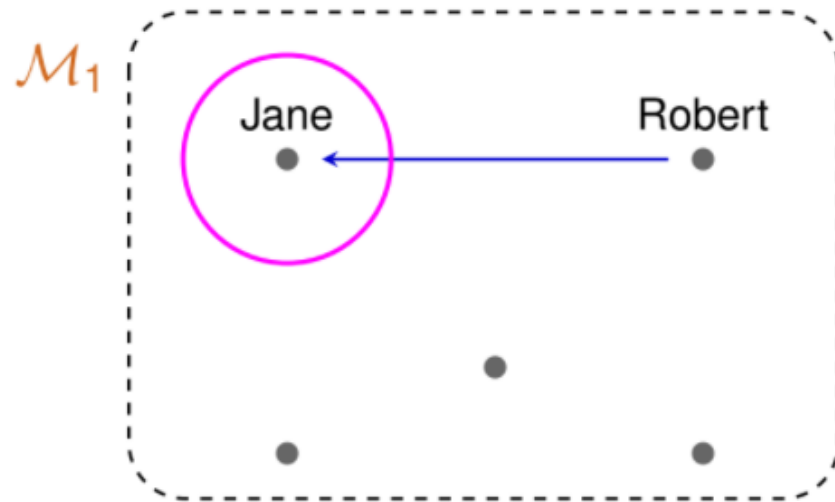
- ▶ $j^{\mathcal{M}_1} = \text{Jane}$, $r^{\mathcal{M}_1} = \text{Robert}$
- ▶ $L^{\mathcal{M}_1} = \{\langle \text{Robert}, \text{Jane} \rangle\}$
 - ▶ Robert loves Jane (blue arrow)
 - ▶ nobody else loves anybody
- ▶ $C^{\mathcal{M}_1} = \{\text{Jane}\}$
 - ▶ Jane is clever (pink circle)
 - ▶ nobody else is clever

For $\mathcal{M}_1$ we find:

$$\begin{array}{lll} \mathcal{M}_1 \models C(j) & \mathcal{M}_1 \models L(r, j) & \mathcal{M}_1 \models C(j) \wedge L(r, j) \\ \mathcal{M}_1 \not\models C(r) & \mathcal{M}_1 \not\models \forall x C(x) & \mathcal{M}_1 \models \neg \forall x C(x) \end{array}$$

$$\mathcal{M}_1 \not\models L(j, r) \quad \mathcal{M}_1 \not\models \exists x C(x)$$

Models



Here we are in a hypothetical world (model $\mathcal{M}_1$) where:

$\mathcal{M}_2 \rightarrow$ (second model)

- ▶ $j^{\mathcal{M}_1} = \text{Jane}$, $r^{\mathcal{M}_1} = \text{Robert}$
- ▶ $L^{\mathcal{M}_1} = \{\langle \text{Robert}, \text{Jane} \rangle\}$
 - ▶ Robert loves Jane (blue arrow)
 - ▶ nobody else loves anybody
- ▶ $C^{\mathcal{M}_1} = \{\text{Jane}\}$
 - ▶ Jane is clever (pink circle)
 - ▶ nobody else is clever

For $\mathcal{M}_1$ we find:

$$\begin{array}{lll} \mathcal{M}_1 \models C(j) & \mathcal{M}_1 \models L(r, j) & \mathcal{M}_1 \models C(j) \wedge L(r, j) \\ \mathcal{M}_1 \not\models C(r) & \mathcal{M}_1 \not\models \forall x C(x) & \mathcal{M}_1 \models \neg \forall x C(x) \end{array}$$

Notation (for truth in a model $\mathcal{M}$)

For a formula ϕ , we denote by $\mathcal{M} \models \phi$ that ϕ holds in $\mathcal{M}$.

$\nexists$ not hold

Models

A *tautology* is **true** in all models

A *contradiction* is **false** in all models

A *contingent* formula is **true** in some model, **false** in another

A *satisfiable* formula is **true** in some model

Models

Semantic entailment $\phi_1, \dots, \phi_n \models \psi$

► in *propositional* logic:

for all valuations v :

$$v(\phi_1) = \text{T} \text{ and } \dots \text{ and } v(\phi_n) = \text{T} \implies v(\psi) = \text{T}$$

► in *predicate* logic:

for all models $\mathcal{M}$:

$$\underbrace{\mathcal{M} \models \phi_1 \text{ and } \dots \text{ and } \mathcal{M} \models \phi_n}_{\text{premise holds}} \implies \underbrace{\mathcal{M} \models \psi}_{\text{conclusion holds}}$$

premise holds conclusion holds

then $\phi_1 \dots \phi_n \models \psi$

Models

Semantic entailment $\phi_1, \dots, \phi_n \models \psi$

► in *propositional* logic:

for all valuations v :

$$v(\phi_1) = \text{T} \text{ and } \dots \text{ and } v(\phi_n) = \text{T} \implies v(\psi) = \text{T}$$

► in *predicate* logic:

for all models $\mathcal{M}$:

$$\mathcal{M} \models \phi_1 \text{ and } \dots \text{ and } \mathcal{M} \models \phi_n \implies \mathcal{M} \models \psi$$

A **counterexample** for a semantic entailment $\phi_1, \dots, \phi_n \models \psi$ in predicate logic consists of a *model* $\mathcal{M}$ with

$$\mathcal{M} \models \phi_1, \dots, \mathcal{M} \models \phi_n \quad \mathcal{M} \not\models \psi$$

Models: Important sem. equiv.

Important semantic equivalences in predicate logic:

$$\neg \exists x \phi \equiv \forall x \neg \phi$$

$$\neg \forall x \phi \equiv \exists x \neg \phi$$

$$\exists x (\phi \vee \psi) \equiv \exists x \phi \vee \exists x \psi$$

$$\forall x (\phi \wedge \psi) \equiv \forall x \phi \wedge \forall x \psi$$

$$\exists x \exists y \phi \equiv \exists y \exists x \phi$$

$$\forall x \forall y \phi \equiv \forall y \forall x \phi$$

Models: Example

Is Socrates a fish ???

Question

Let $F(x)$ represent that x is a fish. Show that

$$\forall x (F(x) \rightarrow M(x)), M(s) \not\models F(s)$$

Let $A = \{p\}$, $F^M = \emptyset$, $M^M = \{p\}$, $s^M = p$

$\forall x (F(x) \rightarrow M(x))$ and $M(s)$

holds in M

but $F(s)$

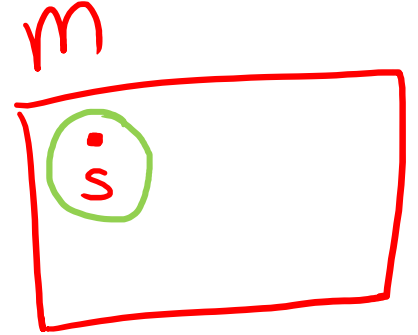
Models: Example

Is Socrates a fish ???

Question

Let $F(x)$ represent that x is a fish. Show that

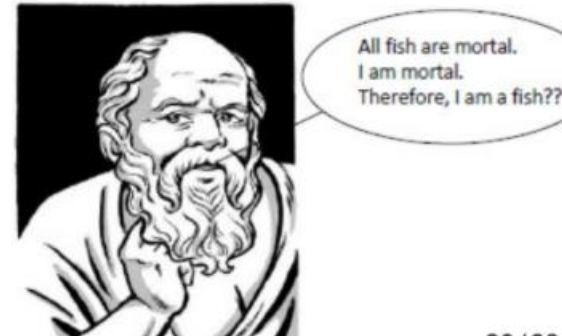
$$\forall x (F(x) \rightarrow M(x)), M(s) \not\models F(s)$$



Answer: Let $A = \{p\}$, $F^M = \emptyset$, $M^M = \{p\}$ and $s^M = p$.
 (Handwritten annotations: "fish" above F^M , "mortal" above M^M , "constant" above s^M)

$\forall x (F(x) \rightarrow M(x))$ and $M(s)$ hold in M .

But $F(s)$ does not.



Exam Question: resit 2020

4. Predicate logic (12 points)

Give two models to show that the following two formulas do not semantically entail each other.

- $\exists x C(x)$
- $\forall x C(x)$

Solution: To prove that the first formula does not semantically entail the second formula, consider a model with elements a, b , where $C(a)$ holds while $C(b)$ does not. The first formula holds on this model, because $C(a)$, but the second formula does not, because $C(b)$ does not hold.

To prove that the second formula does not semantically entail the first formula, consider a model with no elements. The second formula holds on this model, because C holds for all elements (as there are none). But the first formula does not, because there are no elements, so C does not hold for any element.

Models: Exercise!

Exam Question: resit 2021

4. Predicate logic (11 points)

Give two models to show that the formulas $C(x) \rightarrow D(x)$ and $D(x) \rightarrow C(x)$ do not semantically entail each other. (Explain your answer.)

$M_1 = \{a, b\}$ C holds for a , D holds for a, b
not for b , $D(a) \rightarrow C(b)$
 $T \rightarrow F$ does not hold

$M_2 = \{a, b\}$ D holds for a , C holds for a, b
not for b , $C(a) \rightarrow D(b) \Rightarrow$ not holds

Exam Question: resit 2021

4. Predicate logic (11 points)

Give two models to show that the formulas $C(x) \rightarrow D(x)$ and $D(x) \rightarrow C(x)$ do not semantically entail each other. (Explain your answer.)

Solution: As a first model, let $\mathcal{M}_1 = \{a\}$ where $C(a)$ does not hold while $D(a)$ does. If x is interpreted as a , then $C(x) \rightarrow D(x)$ holds in $\mathcal{M}_1$ while $D(x) \rightarrow C(x)$ does not. So the first formula does not semantically entail the second formula.

As a second model, let $\mathcal{M}_2 = \{b\}$ where $C(b)$ holds while $D(b)$ does not. If x is interpreted as b , $C(x) \rightarrow D(x)$ does not hold in $\mathcal{M}_2$ while $D(x) \rightarrow C(x)$ does. So the second formula does not semantically entail the first formula.

Models: Exercise! (extra explanation)

Let's define the predicates $C(x)$ and $D(x)$ as follows:

- $C(x)$: "x is a cat."
- $D(x)$: "x is a dog."

Model 1:

In this model, we have a universe of discourse containing only cats and no dogs. Therefore, every element in the universe satisfies $C(x)$ but does not satisfy $D(x)$. $x = \text{cat}$

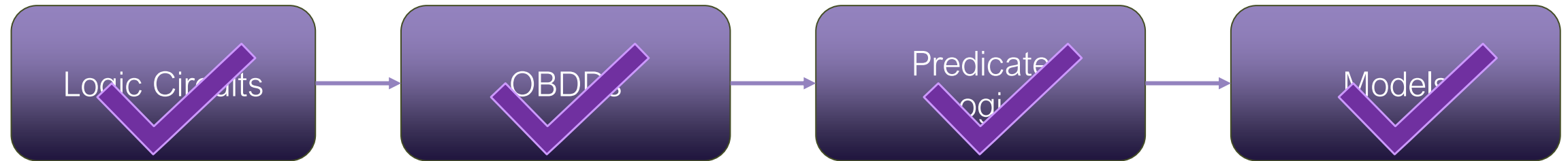
- $C(x) \rightarrow D(x)$: False (since there are instances where $C(x)$ is true and $D(x)$ is false)
- $D(x) \rightarrow C(x)$: True (since there are no instances where $D(x)$ is true and $C(x)$ is false)

Model 2:

In this model, we have a universe of discourse containing only dogs and no cats. Therefore, every element in the universe satisfies $D(x)$ but does not satisfy $C(x)$. $x = \text{dog}$

- $D(x) \rightarrow C(x)$: False (since there are instances where $D(x)$ is true and $C(x)$ is false)
- $C(x) \rightarrow D(x)$: True (since there are no instances where $C(x)$ is true and $D(x)$ is false)

Planning Day 1



Planning

10-15 min break
every 45 min!

Wednesday 	Logic week 5: ○ Logic Circuits Logic week 6: ○ OBDDs Logic week 7: ○ Predicate Logic ○ Models
Thursday	Sets week 5 + 6 + 7

Questions?

- Join the WhatsApp group for slides! Invite link should be in your mail
- If you have any questions, ask them in the WhatsApp group :)